

How does d[k] work?

Programming and Tools for AI – Week 4

James McDermott



Part 1: a lookup table, PairListDict, and OOP

Lookup tables are useful

We've already used `dict` several times. A `dict` acts like a lookup table, aka a map. Here we are mapping animal to number of legs:

```
>>> d = {"cat": 4, "bird": 2, "worm": 0, "dog": 4}
>>> d["cat"]
4
```

Reverse lookup

It's common in AI applications to want both a mapping and the reverse-mapping, eg mapping parent nodes to child nodes in a graph, and vice-versa, so we can traverse in both directions.

In our toy example, map animal to number of legs, and vice-versa.

We carried out a dict **inversion** in a previous week. Recall that something can be lost:

```
>>> d = {"cat": 4, "bird": 2, "worm": 0, "dog": 4}
>>> r = {} # r will be reverse of d
>>> for k, v in d.items():
...     r[v] = k
>>> r[4] # cat is missing!
'dog'
```

Avoid over-writing by storing a list

Idea: with potential key clashes like this, **store a list to avoid over-writing**:

```
>>> r = defaultdict(list)
>>> for k, v in d.items():
...     r[v].append(k)
>>> r[4]
['cat', 'dog']
```

We will use this idea later!

How to implement a lookup table?

Let's pretend we don't have `dict` in the language. Let's implement our own!

- Store a list of `(key, value)` pairs
- Lookup a key: iterate for `k, v in L` and check for matching `k`
- Store a pair: `L.append((key, value))`
- Size: `len(L)`.

All of the above is fine, and it would work. But it would be inconvenient to use, eg, writing a loop every time we want to read a value. (Also very slow, but we'll consider that later.)

A better interface with OOP

We often use **object-oriented programming** to provide a nicer interface. We will now show how to use OOP to mimic the Python `dict` interface (square brackets, `in`, `len`).

The square-bracket notation for reading and writing is implemented like *getters* and *setters* which you might have seen in some other languages.

PairListDict: storing and finding

```
class PairListDict:
    def __init__(self): # d = PairListDict()
        self.pairs = []

    def __setitem__(self, key, value): # d['a'] = 17
        for i, (k, v) in enumerate(self.pairs):
            if k == key: # overwrite existing item
                self.pairs[i] = (key, value)
            return
        self.pairs.append((key, value)) # store new item

    def __getitem__(self, key): # print(d['a'])
        for k, v in self.pairs:
            if k == key:
                return v
        raise KeyError(key)
```


PairListDict: size and iteration

```
def __len__(self): # len(d)
    return len(self.pairs)

def __iter__(self): # for k in d
    for k, v in self.pairs:
        yield k
```

Usage:

```
>>> d = PairListDict()
>>> d["cat"] = 4
>>> d["cat"]
4
>>> len(d)
1
```

Classes and instances

A **class** is a description. An **instance** is one actual thing built to that description.

We used the `class` keyword to create a class `PairListDict` which contains some functions (“methods”) and data.

```
>>> d = PairListDict()           # d is an instance
>>> e = PairListDict()           # e is an instance
>>> d["cat"] = 4
>>> len(d), len(e)
(1, 0)
```

- `class PairListDict:` defines the class.
- `PairListDict()` **calls** the class to create a new instance.
- `d` and `e` share code, but don't share data.

self, and `__init__`

```
def __init__(self):  
    self.pairs = []
```

- `__init__` runs once, automatically, when the instance is built. Its job is to set up the data the instance will need. It's sometimes called the **constructor**.
- `self` is the instance the method was called on. It is passed automatically: you write `d["cat"]`, and Python calls the method with `self = d`.
- `self.pairs` is an **attribute**: a variable that belongs to this instance. A plain `pairs = []` inside `__init__` would be a local variable and would vanish when `__init__` returned.
- Every method takes `self` as its first parameter.

Dunder methods

you write	Python calls
<code>PairListDict()</code>	Build object <code>d</code> and calls <code>d.__init__()</code>
<code>d[k]</code>	<code>d.__getitem__(k)</code>
<code>d[k] = v</code>	<code>d.__setitem__(k, v)</code>
<code>len(d)</code>	<code>d.__len__()</code>
<code>for k in d</code>	<code>d.__iter__()</code>
<code>k in d</code>	<code>d.__contains__(k)</code>
<code>k in d</code>	if <code>__contains__</code> not present: <code>d.__iter__()</code> , and compares each
<code>d</code> (in Python command-line)	<code>d.__repr__()</code>

Methods whose names begin and end with two underscores are **dunder** methods (“double underscore”). Python translates from the nice form on the left to the internal form on the right. Note: REPL is “read-eval-print loop”, eg in `ipython`.

The Python data model

That table is the **Python data model**: a “contract” between objects and the language.

There is no registration step or declaration that `PairListDict` is a dict. If it has `__getitem__`, then `d[k]` works.

```
>>> for k in d:                # works, because we wrote __iter__
...     print(k, d[k])         # works, because we wrote __getitem__
cat 4
```

We can take some code that uses a Python dict, and replace it with our `PairListDict`, and it will work!

“If it walks like a duck, and quacks like a duck, it is a duck.”



Duck typing: suppose we have some object `x` and we try to call `x.quack()`. If `x` has a function `quack`, it works. So for our purposes, `x` is a duck. Python doesn't ask **is `x` really a duck?**

Pause

We learned the basics of Python object-oriented programming.

Our example was about **encapsulation**: we took several algorithms and put them inside a nice dict-like interface. In future weeks we will see a little **polymorphism** and **inheritance** also.

We have solved the original problem, ie creating our own dict-like data structure. But our solution will be **slow**.

Part 2: The problem: computational complexity

Our PairListDict works, but is slow

n	PairListDict	Python's builtin dict
100	0.64	0.01
400	2.27	0.01
1600	9.54	0.01
6400	38.71	0.02

Time (in microseconds) to fill a dictionary with n keys and then look up a key. Our PairListDict gets slow very quickly!

Reading the table

PairListDict: four times the keys, four times slower.

Real Python dict: four times the keys, **almost no change**.

At 6400 keys ours is 2000 times slower.

$O(n)$ and $O(1)$

We write this in **big-O notation**, a statement about how cost (time) grows with the size of the input.

- `PairListDict` is **$O(n)$** — cost grows in proportion to n . Double the data, double the time.
- Python's `dict` is **$O(1)$** — cost does not depend on n at all. Double the data, same time.

In big-O notation, **constants are deliberately ignored**.

A program of speed $O(2n+1000)$ is actually just $O(n)$. Buying a faster computer does not change the big-O complexity.

What does $O(n)$ mean?

If an algorithm is $O(n)$, it means the **number of instructions** executed when you run that algorithm grows **in proportion to n** , the size of the input.

In particular, remember that our `PairListDict` has to **iterate over the keys** to check whether a key is present. If there are n keys, and if checking one key requires m instructions, then the total number of instructions is nm , proportional to n . Here, m is a constant, not related to the size of the input.

Big-O is “worst case”. We might be lucky and find the key in iteration 0, but we still write $O(n)$. Even though the average number of instructions is $nm/2$, that is just $O(n)$.

Where is the time wasted?

Recall PairListDict uses a list internally, so list performance determines our performance.

```
>>> L = ["cat", "dog", "worm"] # a plain list, no dict
>>> L[2] # lookup by position: O(1).
'worm'
>>> "worm" in L # search by value: O(n).
True
```

If we ask for item 2, the list can jump straight to position 2 without touching positions 0 and 1.

If we ask whether "worm" is present, it doesn't know where "worm" would be, so it checks each one.

Our PairListDict always has to **search**; it can never **jump**.

Deleting is $O(n)$ too

We did not write `__delitem__` (delete item), but it would have been slow as well:

- **Find** the pair — $O(n)$, the same search as before.
- **Remove** it — also $O(n)$. A list keeps its items in consecutive positions, so taking one out of the middle means shifting every item after it down by one, to close the gap.
- Total $O(n) + O(n) = O(n)$.

Appending is cheap, $O(1)$, because the `list` knows the location of the final element directly.

In `PairListDict`, several important operations are $O(n)$, which is too slow. It's the wrong data structure, which can't be fixed by a change in code.

Other complexity classes we will see

We will also meet $O(n \log n)$:

- Good sorting algorithms like **quicksort**;

$O(\log n)$:

- Tree-related algorithms such as traversing from a leaf to the root;

$O(n^2)$:

- Comparing every item in a list with every other to create a matrix;
- Bad sorting algorithms like **bubblesort**.

Part 3: hashing and buckets

The idea: let the key say where it lives

Searching is slow because we do not know where to look.

A list is fast when finding `L[2]` because `2` is the location: no need to search.

Idea: **compute a location from the key itself**. Store the pair there. To look it up later, compute the location again from the same key, and go straight to it. No search.

`"cat"` ---> some calculation ---> `17` ---> go to slot 17

Now let's discuss how that calculation can work.

The location must be stable

If we compute a location from the key when we store, and again when we look up, the two answers must agree. Otherwise we put the pair in one place and later go looking in another.

That gives us two hard requirements:

1. **Deterministic.** The same key must give the same answer every time, for the life of the dictionary.
2. **Unchanging.** The key itself must not change. If `[1, 2]` becomes `[1, 2, 3]` while it is in use, the calculation will give a new answer and the old pair is stranded.

Requirement 2 is the reason **a list cannot be a dictionary key**.

What a hash function is

A **hash function** takes any object and returns an `int`.

- Same input gives same output.
- Different inputs *usually* give different outputs, but not always: there are more possible keys than `ints`, so some inputs must share.
- The output looks scrambled. Similar inputs should give unrelated outputs – eg `"cat1"` and `"cat2"` should not end up next to each other.

It is a one-way summary from key to number. We cannot get the key back from the number.

hash in Python

Python has hashing built in:

```
>>> hash(3) # small int hashes to itself, convenient for testing
3
>>> hash(11)
11
>>> hash("cat") # strings hash to large ints; your number will
differ
925243933273910370
>>> hash([1, 2]) # some objects are unhashable
TypeError: unhashable type: 'list'
```

Python enforces Requirement 2 for us, by refusing the key.

From a hash to a slot: the mod rule

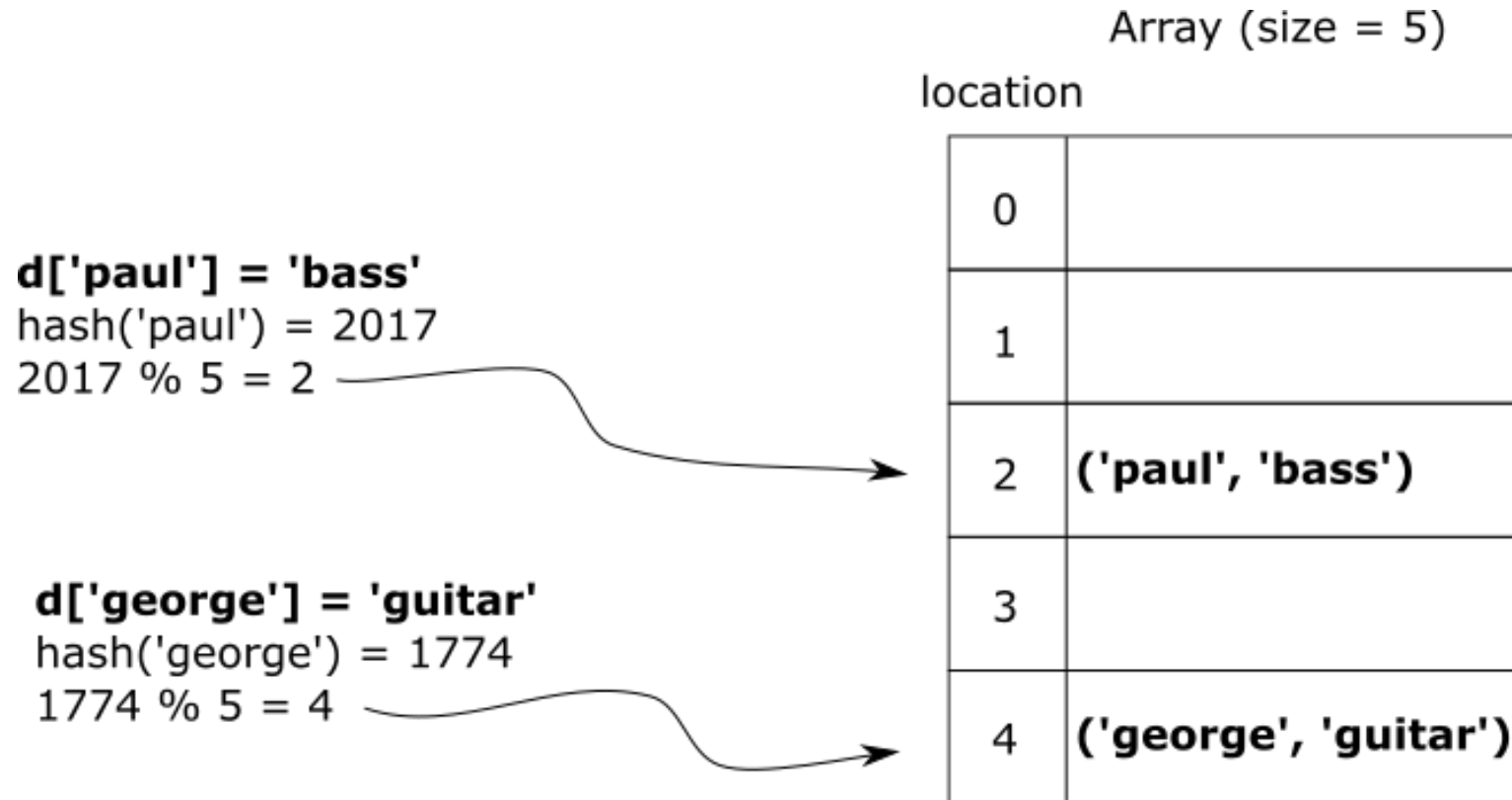
`hash("cat")` is a 19-digit number. We do not have that many slots. We fold it into the range we do have with `%` (the mod or *remainder* operator). Suppose we have a table of 5 slots:

```
>>> hash("cat") % 5 # your number will differ
2
```

`x % 5` is always 0, 1, 2, 3 or 4 — a legal index into a table of 5. Even if `x` is negative, it is mapped to the same range.

```
location = hash(key) % capacity
```

The mod rule, in a picture



Apply hash and mod to find a key's location. `d['paul']` goes straight to slot 2 and `d['george']` straight to slot 4, with no searching.

Two keys, one slot

The picture is missing something. What if two keys land on the same number?

```
>>> hash(3) % 5, hash(8) % 5  
(3, 3)
```

This is a **collision**. With only 5 slots it will happen a lot.

How to handle it: **each slot holds a list of pairs**, not one pair. We call these slots **buckets**.

This is the same idea we used at the start of this week's lecture, about reversing a dict.

Now lookup goes to one bucket and searches *it*. If a bucket gets large, that is a slow list search, so we need to keep the buckets small.

Part 4: Implementation

SimpleDict: a list of buckets

```
class SimpleDict:
    def __init__(self, capacity=8):
        self.capacity = capacity
        self.n = 0
        self.buckets = [[] for _ in range(capacity)]

    def _bucket(self, key):
        return self.buckets[hash(key) % self.capacity]
```

- `self.buckets` is a list of `capacity` empty lists. Note the comprehension.
- `self.n` counts the pairs. `len(self.buckets)` would count the buckets.
- `_bucket` is the core idea. Python users often use a leading underscore to mean “for internal use only”.

Storing a pair

```
def __setitem__(self, key, value):  
    bucket = self._bucket(key)  
    for i, (k, v) in enumerate(bucket):  
        if k == key:  
            bucket[i] = (key, value) # overwrite existing item  
            return  
    bucket.append((key, value)) # add new item  
    self.n += 1  
    if self.n > 0.75 * self.capacity:  
        self._grow()
```

The loop is exactly like `PairListDict.__setitem__`. The difference is the first line: we search **one bucket**, not the whole table.

Finding a pair

```
def __getitem__(self, key):  
    for k, v in self._bucket(key): # fast  
        if k == key:  
            return v  
    raise KeyError(key)
```

Compare it with PairListDict.__getitem__:

```
def __getitem__(self, key):  
    for k, v in self.pairs: # slow  
        if k == key:  
            return v  
    raise KeyError(key)
```

Why it has to grow

Eg, 8 buckets and 8000 keys would give 1000 pairs per bucket, so we would be back to searching: 1000 items. Hashing on its own does not make anything fast — **searching short buckets** makes it fast.

Define the **load factor**: number of pairs divided by number of buckets.

When load factor exceeds (eg) 0.75 we double the number of buckets.

(Now you can see why we need the hash function to distribute keys well.)

Growing

```
def _grow(self):
    pairs = list(self.items())
    self.capacity *= 2
    self.buckets = [[] for _ in range(self.capacity)]
    self.n = 0
    for k, v in pairs:
        self[k] = v
```

We cannot just copy the old buckets across.

A pair's bucket is $\text{hash}(\text{key}) \% \text{capacity}$, and capacity has just changed, so almost every item needs to go to a new location.

It is $O(n)$ to recalculate everything, but we only have to do this rarely, when growing.

The re-inserts cannot set off another grow: capacity just doubled, so the load factor is now about 0.375.

How fast is it now?

n	PairListDict	SimpleDict	real dict
100	0.64	0.10	0.01
400	2.27	0.11	0.01
1600	9.54	0.11	0.01
6400	38.71	0.11	0.02

Timings for lookup given a dictionary with n items. SimpleDict is pretty much flat. The real dict is 10x quicker thanks to optimised C implementation, but that is a **constant factor**. Both columns are $O(1)$.

The trade: space for time

`SimpleDict` holding 1000 keys: 2048 buckets, longest 4, mean 0.49

More than half of all buckets are empty in this report from `SimpleDict`, whereas `PairListDict` wastes nothing.

That is a common trade-off: **spend memory to save time**. The empty buckets are not a flaw, rather they are the consequence of buckets being short on average, which enables fast lookup.

Complexity for the built-in types

operation	list	tuple	dict	set
read $x[i]$, $d[k]$	$O(1)$	$O(1)$	$O(1)$	–
write $x[i]=v$, $d[k]=v$	$O(1)$	–	$O(1)$	–
add one item	$O(1)$	–	$O(1)$	$O(1)$
$\text{len}(x)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
$v \text{ in } x$	$O(n)$	$O(n)$	$O(1)$	$O(1)$
delete one item	$O(n)$	–	$O(1)$	$O(1)$

A set is a dict that ignores values, so set and dict have same complexity. tuple has gaps because it cannot be changed.

A practical speed-up

```
if name in big_list:      #  $O(n)$   
if name in big_set:       #  $O(1)$ 
```

If testing membership repeatedly, and the collection is not tiny, we convert to a `set` (or a `dict`) once and test against that. A one-line change can turn an $O(n^2)$ program into an $O(n)$ one!

Summary

- A dict is a **lookup table**. We created a slow one, and a good one
- **OOP** and the **data model**: write `__getitem__`, `__setitem__`, `__len__`, `__iter__`, and `d[k]`, `len(d)` and `for k in d` all just work.
- Searching a list is **O(n)**. Computing the location from the key with `hash(key) % capacity` is **O(1)**.
- Collisions are normal, so a bucket holds a list; **growing** keeps those lists short, and short lists are what O(1) actually rests on.
- Keys must be **hashable**, because a key whose hash changes is a pair you can never find again.
- The cost was **memory**: half the table is empty, on purpose.

New this week: `class`, `instances`, `self`, `__init__`, `attributes`, `methods`; the data model — `__getitem__`, `__setitem__`, `__len__`, `__contains__`, `__iter__`, `__repr__`; `hash` and `hashability`; `raise`, `KeyError`, `try/except`.